

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 6

«Работа с БД в СУБД MongoDB»

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Коновалова Кира Романовна

Факультет прикладной информатики

Группа К3239

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии 2023

Преподаватель Говорова Марина Михайловна

Санкт-Петербург

2025/2026

Содержание

Цель работы.....	3
Программное обеспечение.....	3
Выполнение работы.....	3
Лабораторная работа 6.1	3
Шаг 1. Установка и первичная проверка MongoDB.....	3
Шаг 2. Создание и удаление базы данных learn	5
Лабораторная работа 6.2	7
Шаг 1. Создание базы learn и заполнение коллекции unicorns	7
Шаг 3. Проекция, порядок добавления и работы с массивами	11
Шаг 4. Логические операторы MongoDB.....	13
Шаг 5. Вложенные объекты: коллекция towns	14
Шаг 6. JavaScripts-функция и курсор.....	16
Шаг 7. Агрегированные запросы.....	18
Шаг 8. Редактирование данных	19
Шаг 9. Удаление данных из коллекции towns	23
Шаг 10. Ссылки между коллекциями	23
Шаг 11. Создание уникального индекса.....	26
Шаг 12. Управление индексами	27
Шаг 13. План запроса и сравнение выполнения с индексами и без индекса.....	28
Ответы на контрольные вопросы	34
Контрольные вопросы по разделу 3.....	34
Контрольные вопросы по разделу 4.....	35
Вывод	36

Цель работы

Цель работы: получить практические навыки установки и запуска MongoDB, а также освоить основные операции работы с документной базой данных: создание базы данных и коллекций, вставку документов, выборку данных, работу с массивами и вложенными объектами, использование курсоров, агрегированные запросы, редактирование и удаление документов, создание ссылок между коллекциями, настройку индексов и анализ плана выполнения запроса

Программное обеспечение

В ходе работы использовались MongoDB Community Server и клиентская оболочка MongoDB Shell (mongosh). Сервер MongoDB был запущен как служба Windows, статус службы — Running. В используемой среде вместо старой клиентской консоли mongo применялась современная оболочка mongosh, поэтому часть команд из методических материалов была выполнена современными аналогами: insertOne/insertMany вместо insert, deleteMany вместо remove, createIndex вместо ensureIndex

Выполнение работы

Лабораторная работа 6.1

Шаг 1. Установка и первичная проверка MongoDB

После установки MongoDB была проверена работа серверной части. В PowerShell была выполнена команда Get-Service MongoDB. В результате служба MongoDB Server имела статус Running, значит сервер базы данных был запущен и готов к подключению.

```
PS C:\WINDOWS\system32> mongosh
Current Mongosh Log ID: 6a3688cf53634e0638abc113
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB:      8.3.4
Using Mongosh:      2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----
  The server generated these startup warnings when booting
  2026-06-20T15:15:10.922+03:00: Access control is not enabled for the database. Read
  and write access to data and configuration is unrestricted
  -----

test> db.version()
8.3.4
test>
```

Далее была запущена оболочка mongosh. После подключения были выполнены команды db.version(), db.help(), db.help и db.stats(). Команда db.version() показала версию сервера

MongoDB, команда `db.help()` вывела список доступных методов объекта `db`, а команда `db.stats()` показала статистику текущей базы данных

```
test> db.help()

Database Class:

  getMongo           Returns the current database connection
  getName            Returns the name of the DB
  getCollectionNames Returns an array containing the names of all collections in the current database.
  getCollectionInfo  Returns an array of documents with collection information, i.e. collection name and options, for the current database.
  runCommand         Run an arbitrary command on the database.
  adminCommand       Run a specified admin/diagnostic pipeline which does not require an underlying collection.
  aggregate          Returns another database without modifying the db variable in the shell environment.
  getSiblingDB       Returns a collection or a view object that is functionally equivalent to using the db.<collectionName>.
  getCollection      Removes the current database, deleting the associated data files.
  dropDatabase       Creates a new user for the database on which the method is run. db.createUser() returns a duplicate user error if the user already e
  exists on the database.
  updateUser         Updates the user's profile on the database on which you run the method. An update to a field completely replaces the previous field'
  s values. This includes updates to the user's roles array.
  changeUserPassword Updates a user's password. Run the method in the database where the user is defined, i.e. the database you created the user.
  logout            Ends the current authentication session. This function has no effect if the current session is not authenticated.
  dropUser          Removes the user from the current database.
  dropAllUsers       Removes all users from the current database.
  auth              Allows a user to authenticate to the database from within the shell.
  grantRolesToUser   Grants additional roles to a user.
  revokeRolesFromUser Removes a one or more roles from a user on the current database.
  getUser           Returns user information for a specified user. Run this method on the user's database. The user must exist on the database on which
  the method runs.
  getUsers           Returns information for all the users in the database.
```

```
test> db.help

Database Class:

  getMongo           Returns the current database connection
  getName            Returns the name of the DB
  getCollectionNames Returns an array containing the names of all collections in the current database.
  getCollectionInfo  Returns an array of documents with collection information, i.e. collection name and options, for the current database.
  runCommand         Run an arbitrary command on the database.
  adminCommand       Run a specified admin/diagnostic pipeline which does not require an underlying collection.
  aggregate          Returns another database without modifying the db variable in the shell environment.
  getSiblingDB       Returns a collection or a view object that is functionally equivalent to using the db.<collectionName>.
  getCollection      Removes the current database, deleting the associated data files.
  dropDatabase       Creates a new user for the database on which the method is run. db.createUser() returns a duplicate user error if the user already e
  exists on the database.
  updateUser         Updates the user's profile on the database on which you run the method. An update to a field completely replaces the previous field'
  s values. This includes updates to the user's roles array.
  changeUserPassword Updates a user's password. Run the method in the database where the user is defined, i.e. the database you created the user.
  logout            Ends the current authentication session. This function has no effect if the current session is not authenticated.
  dropUser          Removes the user from the current database.
  dropAllUsers       Removes all users from the current database.
  auth              Allows a user to authenticate to the database from within the shell.
  grantRolesToUser   Grants additional roles to a user.
  revokeRolesFromUser Removes a one or more roles from a user on the current database.
  getUser           Returns user information for a specified user. Run this method on the user's database. The user must exist on the database on which
  the method runs.
```

```
test> db.stats()
{
  db: 'test',
  collections: Long('0'),
  views: Long('0'),
  objects: Long('0'),
  avgObjSize: 0,
  dataSize: 0,
  storageSize: 0,
  indexes: Long('0'),
  indexSize: 0,
  totalSize: 0,
  scaleFactor: Long('1'),
  fsUsedSize: 0,
  fsTotalSize: 0,
  ok: 1
}
test> |
```

В методических материалах указано, что при вызове `db.help` без скобок может выводиться внутреннее представление функции. В используемой версии `mongosh` оба варианта вывели справочную информацию. Это связано с особенностями современной оболочки, но результат показывает, что клиент успешно работает и позволяет просматривать методы объекта `db`

Шаг 2. Создание и удаление базы данных learn

Для создания базы данных была выполнена команда `use learn`. В MongoDB база данных фактически создается не в момент переключения, а после добавления первого документа. Поэтому после команды `use learn` база была выбрана, но в списке `show dbs` могла появиться только после вставки данных

```
test> use learn
switched to db learn
learn> db
learn
learn> show dbs
admin    40.00 KiB
config  92.00 KiB
local   40.00 KiB
learn>
```

Затем была создана коллекция unicorns путем вставки первого документа.

```
db.unicorns.insertOne({name: 'Aurora', gender: 'f', weight: 450})
```

```
learn> db.unicorns.insertOne({name: 'Aurora', gender: 'f', weight: 450})
{
  acknowledged: true,
  insertedId: ObjectId('6a368ba053634e0638abc114')
}
learn> show collections
unicorns
```

После этого было проверено содержимое коллекции, список коллекций, выполнено переименование коллекции, просмотр статистики, удаление коллекции и удаление базы данных learn.

```
learn> db.unicorns.renameCollection("unicorns_old")
{ ok: 1 }
learn> show collections
unicorns_old
learn>
```

[illegible]

```
    }  
  },  
  sharded: false,  
  size: 65,  
  count: 1,  
  numOrphanDocs: 0,  
  storageSize: 20480,  
  totalIndexSize: 20480,  
  totalSize: 40960,  
  indexSizes: { _id_: 20480 },  
  avgObjSize: 65,  
  ns: 'learn.unicorns_old',  
  nindexes: 1,  
  scaleFactor: 1  
}
```

```
learn> db.unicorns_old.drop()  
true  
learn> show collections  
  
learn> db.dropDatabase()  
{ ok: 1, dropped: 'learn' }  
learn> show dbs  
admin    40.00 KiB  
config   92.00 KiB  
local    40.00 KiB  
learn> |
```

Лабораторная работа 6.2

Шаг 1. Создание базы learn и заполнение коллекции unicorns

В основной части лабораторной работы база данных learn была создана заново. Для чистого выполнения коллекция unicorns была предварительно очищена, после чего в нее были добавлены документы с данными о единорогах. Основной набор документов был добавлен методом insertMany, так как он позволяет вставить сразу несколько документов одной командой.

```
use learn
db.unicorns.drop()
db.unicorns.insertMany([
  {name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63},
  {name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43},
  {name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182},
  {name: 'Rooodooles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99},
  {name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80},
  {name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40},
  {name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39},
  {name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2},
  {name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33},
  {name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54},
  {name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'}
])
```

Документ единорога Dunx был добавлен вторым способом: сначала объект был сохранен в переменную document, а затем вставлен в коллекцию методом insertOne.

```
document = {
  name: 'Dunx',
  loves: ['grape', 'watermelon'],
  weight: 704,
  gender: 'm',
  vampires: 165
}
db.unicorns.insertOne(document)
```

```
learn> document = {
  name: 'Dunx',
  loves: ['grape', 'watermelon'],
  weight: 704,
  gender: 'm',
  vampires: 165
}
{
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
learn> db.unicorns.insertOne(document)
{
  acknowledged: true,
  insertedId: ObjectId('6a3aa496604594cf4cabc11f')
}
learn> |
```

После вставки было проверено содержимое коллекции и количество документов. Команда `countDocuments()` вернула 12, следовательно, все документы были добавлены корректно

```
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a3aa407604394cf4cabc110'),
    name: 'Hunny',
    loves: [ 'carrot', 'papaya' ],
    weight: 688,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a3aa407604394cf4cabc116'),
    name: 'Hunra',
    loves: [ 'carrot', 'grape' ],
    weight: 458,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a3aa407604394cf4cabc11a'),
    name: 'Unicornus',
    loves: [ 'energy', 'redbull' ],
    weight: 904,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a3aa407604394cf4cabc117'),
    name: 'Runnoodlas',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'a',
    vampires: 99
  },
  {
    _id: ObjectId('6a3aa407604394cf4cabc112'),
    name: 'Sulnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 509,
    gender: 'f',
    vampires: 89
  },
  {
    _id: ObjectId('6a3aa407604394cf4cabc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 49
  },
  {
    _id: ObjectId('6a3aa407604394cf4cabc11a'),
    name: 'Hunny',
    loves: [ 'grape', 'lemon' ],
    weight: 699,
    gender: 'a',
    vampires: 39
  },
  {
    _id: ObjectId('6a3aa407604594cf4cabc11b'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a3aa407604594cf4cabc11c'),
    name: 'Lein',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a3aa407604594cf4cabc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 659,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a3aa407604594cf4cabc11e'),
    name: 'Mimoo',
    loves: [ 'grape', 'carrot' ],
    weight: 549,
    gender: 'f'
  },
  {
    _id: ObjectId('6a3aa496604594cf4cabc11f'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  }
]
learn> db.unicorns.countDocuments()
12
learn> |
```


Шаг 2. Выборка данных из коллекции unicorns

Далее были выполнены запросы на выборку данных. Сначала были сформированы списки самцов и самок единорогов. Для сортировки результата по имени использовался метод `sort({name: 1})`.

```
db.unicorns.find({gender: 'm'}).sort({name: 1})
```

```
learn> db.unicorns.find({gender: 'm'}).sort({name: 1})
[
  {
    _id: ObjectId('6a3aa496604594cf4cabc11f'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a3aa407604594cf4cabc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a3aa497604594cf4cabc11a'),
    name: 'Menny',
    loves: [ 'grape', 'lemon' ],
    weight: 696,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a3aa407604594cf4cabc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a3aa407604594cf4cabc11b'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a3aa407604594cf4cabc117'),
    name: 'Hoooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a3aa407604594cf4cabc116'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  }
]
```

```
db.unicorns.find({gender: 'f'}).sort({name: 1})
```

```
learn> db.unicorns.find({gender: 'F'}).sort({name: 1})
[
  {
    _id: ObjectId('6a3aa487684594cf4cabc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'F',
    vampires: 43
  },
  {
    _id: ObjectId('6a3aa487684594cf4cabc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'F',
    vampires: 48
  },
  {
    _id: ObjectId('6a3aa487684594cf4cabc11c'),
    name: 'Leila',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'F',
    vampires: 33
  },
  {
    _id: ObjectId('6a3aa487684594cf4cabc11e'),
    name: 'Mimmi',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'F'
  },
  {
    _id: ObjectId('6a3aa487684594cf4cabc118'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'F',
    vampires: 88
  }
]
learn> |
```

Список самок был ограничен первыми тремя документами с помощью `limit(3)`. Это позволило получить только первые три записи после сортировки по имени.

```
learn> db.unicorns.find({gender: 'F'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a3aa487684594cf4cabc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'F',
    vampires: 43
  },
  {
    _id: ObjectId('6a3aa487684594cf4cabc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'F',
    vampires: 48
  },
  {
    _id: ObjectId('6a3aa487684594cf4cabc11c'),
    name: 'Leila',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'F',
    vampires: 33
  }
]
learn> |
```

Затем были найдены самки, которые любят carrot. Поле `loves` является массивом, но MongoDB позволяет искать документ по отдельному значению внутри массива. Для ограничения результата одной особью были использованы два способа: `findOne()` и `find().limit(1)`

```

learn> db.unicorns.find({gender: 'f', loves: 'carrot'})
[
  {
    _id: ObjectId('6a3aa876d4594ef4cabc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a3aa876d4594ef4cabc116'),
    name: 'Solmara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 38
  },
  {
    _id: ObjectId('6a3aa876d4594ef4cabc11e'),
    name: 'Ramon',
    loves: [ 'grape', 'carrot' ],
    weight: 500,
    gender: 'f'
  }
]
learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('6a3aa876d4594ef4cabc115'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
[
  {
    _id: ObjectId('6a3aa876d4594ef4cabc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn>

```

Шаг 3. Проекция, порядок добавления и работы с массивами

Для вывода списка самцов без информации о поле и предпочтениях была использована проекция. В проекции значение 0 означает, что поле нужно исключить из результата. Также было исключено поле `_id`, чтобы результат был компактнее.

```
db.unicorns.find({gender: 'm'}, {_id: 0, loves: 0, gender: 0}).sort({name: 1})
```

```

learn> db.unicorns.find(
  {
    gender: 'm',
    {loves: 0, gender: 0}
  }, sort({name: 1})
)
[
  {
    _id: ObjectId('6a3aa86d4c594ef4cabc114'),
    name: 'Dana',
    weight: 700,
    vampires: 100
  },
  {
    _id: ObjectId('6a3aa876d4594ef4cabc11d'),
    name: 'Harry',
    weight: 600,
    vampires: 61
  },
  {
    _id: ObjectId('6a3aa876d4594ef4cabc11a'),
    name: 'Henny',
    weight: 600,
    vampires: 39
  },
  {
    _id: ObjectId('6a3aa876d4594ef4cabc11d'),
    name: 'Pasha',
    weight: 650,
    vampires: 54
  },
  {
    _id: ObjectId('6a3aa876d4594ef4cabc11b'),
    name: 'Galvigh',
    weight: 421,
    vampires: 2
  },
  {
    _id: ObjectId('6a3aa876d4594ef4cabc117'),
    name: 'Brazendallies',
    weight: 575,
    vampires: 99
  },
  {
    _id: ObjectId('6a3aa876d4594ef4cabc11c'),
    name: 'Unicorn',
    weight: 500,
    vampires: 102
  }
]
learn>

```

Для вывода единорогов в обратном порядке добавления использовалась сортировка по \$natural: -1. Первым в результате отображается Dux, так как он был добавлен последним.

```
db.unicorns.find().sort({$natural: -1})
```

```
learn> db.unicorns.find().sort({$natural: -1})
[
  {
    _id: ObjectId('6a1aa408804294c79c8b11f'),
    name: 'Dux',
    loves: [ 'grape', 'watermelon' ],
    weight: 984,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a1aa407b04294c79c8b11e'),
    name: 'Mimue',
    loves: [ 'grape', 'carrot' ],
    weight: 649,
    gender: 'f'
  },
  {
    _id: ObjectId('6a1aa407b04294c79c8b11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 656,
    gender: 'm',
    vampires: 34
  },
  {
    _id: ObjectId('6a1aa407b04294c79c8b11c'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 11
  },
  {
    _id: ObjectId('6a1aa407a04294c79c8b11b'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 7
  },
  {
    _id: ObjectId('6a1aa40704294c79c8b11a'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 696,
    gender: 'm',
    vampires: 10
  },
  {
    _id: ObjectId('6a1aa40704294c79c8b119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 731,
    gender: 'f',
    vampires: 48
  },
  {
    _id: ObjectId('6a1aa40704294c79c8b118'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 731,
    gender: 'f',
    vampires: 48
  }
]
```

Для вывода первого любимого предпочтения использовался оператор \$slice. Он позволяет выводить не весь массив, а только указанное количество элементов. В данном случае у каждого единорога выводилось имя и первый элемент массива loves.

```
db.unicorns.find({}, {_id: 0, name: 1, loves: {$slice: 1}})
```

```
learn> db.unicorns.find(
  {
    {
      _id: 0, name: 1, loves: {$slice: 1}
    }
  }
)
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicron', loves: [ 'energon' ] },
  { name: 'Rococooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Mimue', loves: [ 'grape' ] },
  { name: 'Dux', loves: [ 'grape' ] }
]
learn> |
```

Шаг 4. Логические операторы MongoDB

В следующем блоке были использованы логические операторы MongoDB. Для поиска самок весом от 500 до 700 кг применялись операторы \$gte и \$lte. Полтонны соответствует 500 кг.

```
db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
```

```
learn> db.unicorns.find(
  {
    gender: 'f',
    weight: {$gte: 500, $lte: 700}
  },
  { _id: 0 }
)
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 548,
    gender: 'f'
  }
]
learn> |
```

Для поиска самцов весом от 500 кг, которые одновременно предпочитают grape и lemon, использовался оператор \$all. Он проверяет, что в массиве присутствуют все перечисленные значения. По данным коллекции этому условию соответствует Kenny.

```
db.unicorns.find({gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}}, {_id: 0})
```

```
learn> db.unicorns.find(
  {
    gender: 'm',
    weight: {$gte: 500},
    loves: {$all: ['grape', 'lemon']}
  },
  { _id: 0 }
)
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
learn> |
```

Для поиска документов без поля vampires применялся оператор \$exists со значением false. В результате был найден единорог Nimue, у которого поле vampires не было указано при вставке.

```
db.unicorns.find({vampires: {$exists: false}}, {_id: 0})
```

```
learn> db.unicorns.find(
|   {vampires: {$exists: false}},
|   {_id: 0}
| )
|
| [
|   {
|     name: 'Nimue',
|     loves: [ 'grape', 'carrot' ],
|     weight: 540,
|     gender: 'f'
|   }
| ]
learn> |
```

В последнем запросе были объединены фильтрация, сортировка и \$slice: были выведены имена самцов в алфавитном порядке и только первое значение из массива loves.

```
db.unicorns.find({gender: 'm'}, {_id: 0, name: 1, loves: {$slice: 1}}).sort({name: 1})
```

```
learn> db.unicorns.find(
|   {gender: 'm'},
|   {_id: 0, name: 1, loves: {$slice: 1}}
| ).sort({name: 1})
|
| [
|   { name: 'Dunx', loves: [ 'grape' ] },
|   { name: 'Horny', loves: [ 'carrot' ] },
|   { name: 'Kenny', loves: [ 'grape' ] },
|   { name: 'Pilot', loves: [ 'apple' ] },
|   { name: 'Raleigh', loves: [ 'apple' ] },
|   { name: 'Rooooooodles', loves: [ 'apple' ] },
|   { name: 'Unicrom', loves: [ 'energon' ] }
| ]
```

Шаг 5. Вложенные объекты: коллекция towns

Для изучения запросов к вложенным объектам была создана коллекция towns. Каждый документ описывает город, а информация о мэре хранится во вложенном объекте mayor.

```
db.towns.drop()
db.towns.insertMany([
  {
    name: "Punxsutawney",
    population: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: ["phil the groundhog"],
    mayor: {name: "Jim Wehrle"}
  },
  {
    name: "New York",
    population: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["statue of liberty", "food"],
    mayor: {name: "Michael Bloomberg", party: "I"}
```

```

    },
    {
      name: "Portland",
      population: 528000,
      last_sensus: ISODate("2009-07-20"),
      famous_for: ["beer", "food"],
      mayor: {name: "Sam Adams", party: "D"}
    }
  ])
db.towns.find()

```

```

learn> db.towns.drop()
true
learn> db.towns.insertMany([
  {
    name: "Punkstamney",
    population: 6280,
    last_sensus: ISODate("2008-01-31"),
    famous_for: ["phil the groundhog"],
    mayor: {
      name: "Jim Wehrle"
    }
  },
  {
    name: "New York",
    population: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["statue of liberty", "food"],
    mayor: {
      name: "Michael Bloomberg",
      party: "I"
    }
  },
  {
    name: "Portland",
    population: 528000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: {
      name: "Sam Adams",
      party: "D"
    }
  }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a3aa8e3604594cf4cabc120'),
    '1': ObjectId('6a3aa8e3604594cf4cabc121'),
    '2': ObjectId('6a3aa8e3604594cf4cabc122')
  }
}
learn>

```

```

learn> db.towns.find()
[
  {
    _id: ObjectId('6a3aa8e3604594cf4cabc120'),
    name: 'Punkstamney',
    population: 6280,
    last_sensus: ISODate('2008-01-31T00:00:00.000Z'),
    famous_for: [ 'phil the groundhog' ],
    mayor: { name: 'Jim Wehrle' }
  },
  {
    _id: ObjectId('6a3aa8e3604594cf4cabc121'),
    name: 'New York',
    population: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'statue of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a3aa8e3604594cf4cabc122'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
learn>

```

Для обращения к полю внутри вложенного объекта использовался оператор точка. Например, поле party внутри объекта mayor записывается как "mayor.party". С помощью такого запроса был найден город с независимым мэром, у которого party равно "I".

```
learn> db.towns.find(
|   {"mayor.party": "I"},
|   {_id: 0, name: 1, mayor: 1}
| )
[
|   { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
| ]
```

Для поиска беспартийных мэров использовался оператор \$exists. В данном случае ищались документы, где поле mayor.party отсутствует.

```
learn> db.towns.find(
|   {"mayor.party": {$exists: false}},
|   {_id: 0, name: 1, mayor: 1}
| )
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
```

Шаг 6. JavaScripts-функция и курсор

MongoDB Shell позволяет использовать элементы JavaScript. Была создана функция для отбора самцов единорогов. В современной версии mongosh основной рабочий запрос был выполнен через стандартный фильтр {gender: "m"}.

```
fn = function() {
  return this.gender == "m";
}
db.unicorns.find({gender: "m"})
```



```

learn> fn = function() {
|   return this.gender == "m";
| }
[Function: fn]
learn> db.unicorns.find(fn)
MongoInvalidArgumentError: Query filter must be a plain object or ObjectId
learn> db.unicorns.find({gender: "m"})
[
  {
    _id: ObjectId('6a3aa4b70d04594cf4cabc11a'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 688,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a3aa4b70d04594cf4cabc11b'),
    name: 'Unicorn',
    loves: [ 'amergon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a3aa4b70d04594cf4cabc117'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 579,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a3aa4b70d04594cf4cabc11a'),
    name: 'Horny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 19
  },
  {
    _id: ObjectId('6a3aa4b70d04594cf4cabc11b'),
    name: 'Balaligh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a3aa4b70d04594cf4cabc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 34
  },
  {
    _id: ObjectId('6a3aa4b70d04594cf4cabc11f'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 784,
    gender: 'm',
    vampires: 165
  }
]
learn>

```

Далее был создан курсор для первых двух самцов после сортировки по имени. Курсор хранит результат выборки и позволяет последовательно обработать документы. Для вывода имен использовался метод `forEach`.

```

var cursor = db.unicorns.find({gender: "m"}).sort({name: 1}).limit(2)
cursor.forEach(function(obj) {
  print(obj.name)
})

```

```

learn> var cursor = db.unicorns.find({gender: "m"}).sort({name: 1}).limit(2)

learn> cursor.forEach(function(obj) {
|   print(obj.name)
| })
Dunx
Horny

```

Шаг 7. Агрегированные запросы

Для подсчета количества самок весом от 500 до 600 кг использовался метод `countDocuments()`. В результате было найдено 2 документа: Solnara и Nimue.

```
db.unicorns.countDocuments({gender: 'f', weight: {$gte: 500, $lte: 600}})
```

```
learn> db.unicorns.countDocuments({
|   gender: 'f',
|   weight: {$gte: 500, $lte: 600}
| })
2
learn> |
```

Для вывода уникального списка предпочтений использовался метод `distinct()`. Так как поле `loves` является массивом, MongoDB учитывает отдельные элементы этого массива.

```
db.unicorns.distinct("loves")
```

```
learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
learn> |
```

Для подсчета количества особей каждого пола использовался метод `aggregate()` с группировкой по полю `gender`. Оператор `$sum` увеличивал счетчик на 1 для каждого документа в группе.

```
db.unicorns.aggregate([
  {
    $group: {
      _id: "$gender",
      count: {$sum: 1}
    }
  }
])
```

```
learn> db.unicorns.aggregate([
|   {
|     $group: {
|       _id: "$gender",
|       count: {$sum: 1}
|     }
|   }
| ])
[ { _id: 'm', count: 7 }, { _id: 'f', count: 5 } ]
learn> |
```

Шаг 8. Редактирование данных

В разделе редактирования данных были выполнены разные виды обновлений документов. Сначала в коллекцию был добавлен новый единорог Barny.

```
db.unicorns.insertOne({
  name: 'Barny',
  loves: ['grape'],
  weight: 340,
  gender: 'm'
})
```

```
learn> db.unicorns.insertOne({
|   name: 'Barny',
|   loves: ['grape'],
|   weight: 340,
|   gender: 'm'
| })
| {
|   acknowledged: true,
|   insertedId: ObjectId('6a3aab6f604594cf4cabc123')
| }
```

Для единорога Айна были изменены вес и количество убитых вампиров. Для этого использовался оператор \$set, который обновляет только указанные поля.

```
db.unicorns.updateOne(
  {name: 'Ayna'},
  {$set: {weight: 800, vampires: 51}}
)
db.unicorns.find({name: 'Ayna'}, {_id: 0})
```

```
learn> db.unicorns.updateOne(
|   {name: 'Ayna'},
|   {
|     $set: {
|       weight: 800,
|       vampires: 51
|     }
|   }
| )
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 1,
|   modifiedCount: 1,
|   upsertedCount: 0
| }
learn> db.unicorns.find(
|   {name: 'Ayna'},
|   {_id: 0}
| )
| [
|   {
|     name: 'Ayna',
|     loves: ['strawberry', 'lemon'],
|     weight: 800,
|     gender: 'f',
|     vampires: 51
|   }
| ]
```

Для Raleigh в массив loves было добавлено значение redbull. Был использован оператор \$addToSet, так как он добавляет значение только в том случае, если его еще нет в массиве.

```
db.unicorns.updateOne(
  {name: 'Raleigh'},
  {$addToSet: {loves: 'redbull'}}
)
db.unicorns.find({name: 'Raleigh'}, {_id: 0})
```



```
learn> db.unicorns.updateOne(
| {name: 'Raleigh'},
| {
|   $addToSet: {
|     loves: 'redbull'
|   }
| }
| )
|
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 1,
|   modifiedCount: 1,
|   upsertedCount: 0
| }
learn> db.unicorns.find(
| {name: 'Raleigh'},
| {_id: 0}
| )
|
| [
|   {
|     name: 'Raleigh',
|     loves: [ 'apple', 'sugar', 'redbull' ],
|     weight: 421,
|     gender: 'm',
|     vampires: 2
|   }
| ]
```

Всем самцам было увеличено количество убитых вампиров на 5. Для этого использовались updateMany() и оператор \$inc. Команда updateMany() была выбрана потому, что нужно было изменить сразу все документы, подходящие под условие gender: "m".

```
db.unicorns.updateMany(
  {gender: 'm'},
  {$inc: {vampires: 5}}
)
db.unicorns.find(
  {gender: 'm'},
  {_id: 0, name: 1, gender: 1, vampires: 1}
).sort({name: 1})
```

```

learn> db.unicorns.updateMany(
  {gender: 'm'},
  {
    $inc: {
      vampires: 5
    }
  }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 8,
  modifiedCount: 8,
  upsertedCount: 0
}
learn> db.unicorns.find(
  {gender: 'm'},
  {_id: 0, name: 1, gender: 1, vampires: 1}
).sort({name: 1})
[
  { name: 'Barney', gender: 'm', vampires: 5 },
  { name: 'Dunx', gender: 'm', vampires: 170 },
  { name: 'Horny', gender: 'm', vampires: 68 },
  { name: 'Kenny', gender: 'm', vampires: 44 },
  { name: 'Pilot', gender: 'm', vampires: 59 },
  { name: 'Maleigh', gender: 'm', vampires: 7 },
  { name: 'Ropeooodles', gender: 'm', vampires: 104 },
  { name: 'Unicorn', gender: 'm', vampires: 187 }
]

```

Для города Portland поле mayor.party было удалено с помощью оператора \$unset. После этого мэр Portland стал беспартийным, так как в документе больше нет поля party.

```

db.towns.updateOne(
  {name: 'Portland'},
  {$unset: {"mayor.party": ""}}
)
db.towns.find({name: 'Portland'}, {_id: 0, name: 1, mayor: 1})

```

```

learn> db.towns.updateOne(
  {name: 'Portland'},
  {
    $unset: {
      "mayor.party": ""
    }
  }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.towns.find(
  {name: 'Portland'},
  {_id: 0, name: 1, mayor: 1}
)
[ { name: 'Portland', mayor: { name: 'Sam Adams' } } ]

```

Для Pilot в массив loves было добавлено значение chocolate. Для этого использовался оператор \$push, который добавляет новое значение в массив.

```

db.unicorns.updateOne(
  {name: 'Pilot'},
  {$push: {loves: 'chocolate'}}
)
db.unicorns.find({name: 'Pilot'}, {_id: 0})

```

```

learn> db.unicorns.updateOne(
|   {name: 'Pilot'},
|   {
|     $push: {
|       loves: 'chocolate'
|     }
|   }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find(
|   {name: 'Pilot'},
|   {_id: 0}
| )
[
  {
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]

```

Для Aurora были добавлены sugar и lemon. Здесь использовались \$addToSet и \$each: \$each позволяет добавить несколько значений сразу, а \$addToSet защищает от повторов.

```

db.unicorns.updateOne(
  {name: 'Aurora'},
  {$addToSet: {loves: {$each: ['sugar', 'lemon']}}}
)
db.unicorns.find({name: 'Aurora'}, {_id: 0})

```

```

learn> db.unicorns.updateOne(
|   {name: 'Aurora'},
|   {
|     $addToSet: {
|       loves: {
|         $each: ['sugar', 'lemon']
|       }
|     }
|   }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find(
|   {name: 'Aurora'},
|   {_id: 0}
| )
[
  {
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]

```

Шаг 9. Удаление данных из коллекции towns

В этом разделе были выполнены операции удаления данных из коллекции towns. Сначала с помощью запроса к вложенному полю "mayor.party" были найдены города, у которых у мэра отсутствует поле party. Затем методом deleteMany() были удалены все документы, соответствующие этому условию.

```
db.towns.find({}, {_id: 0, name: 1, mayor: 1})
db.towns.deleteMany({"mayor.party": {$exists: false}})
db.towns.find({}, {_id: 0, name: 1, mayor: 1})
```

После удаления в коллекции остался только New York, потому что у его мэра поле party присутствовало. Далее коллекция была очищена полностью командой `deleteMany({})`, после чего количество документов стало равно 0. Команда `show collections` позволила проверить список доступных коллекций в текущей базе данных.

```
db.towns.deleteMany({})
db.towns.countDocuments()
show collections
```

```
learn> db.towns.find(
|   {},
|   { _id: 0, name: 1, mayor: 1 }
| )
|
| [
|   { name: 'Punkstammy', mayor: { name: 'Jim Wehrle' } },
|   { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } },
|   { name: 'Portland', mayor: { name: 'San Adams' } }
| ]
learn> db.towns.deleteMany({
|   "mayor.party": { $exists: false }
| })
{ acknowledged: true, deletedCount: 2 }
learn> db.towns.find(
|   {},
|   { _id: 0, name: 1, mayor: 1 }
| )
|
| [
|   { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
| ]
learn> db.towns.deleteMany({})
{ acknowledged: true, deletedCount: 1 }
learn> db.towns.find()

learn> show collections
towns
unicorns
```

Шаг 10. Ссылки между коллекциями

Для изучения ссылок была создана отдельная коллекция `habitats`, содержащая зоны обитания единорогов. Для каждой зоны был задан краткий идентификатор `_id`, полное название и описание.

```
db.habitats.drop()
db.habitats.insertMany([
  {
    _id: "forest",
    name: "Magic Forest",
    description: "A quiet forest habitat with trees, berries and fresh water."
```

```

    },
    {
      _id: "mountains",
      name: "Crystal Mountains",
      description: "A cold mountain habitat with rocks, caves and clean air."
    },
    {
      _id: "valley",
      name: "Sunny Valley",
      description: "A warm valley habitat with grass, flowers and open space."
    }
  ]
)
db.habitats.find()

```

```

learn> db.habitats.drop()
true
learn> db.habitats.insertMany([
  {
    _id: "forest",
    name: "Magic Forest",
    description: "A quiet forest habitat with trees, berries and fresh water."
  },
  {
    _id: "mountains",
    name: "Crystal Mountains",
    description: "A cold mountain habitat with rocks, caves and clean air."
  },
  {
    _id: "valley",
    name: "Sunny Valley",
    description: "A warm valley habitat with grass, flowers and open space."
  }
])
{
  acknowledged: true,
  insertedIds: { '0': 'forest', '1': 'mountains', '2': 'valley' }
}
learn> db.habitats.find()
[
  {
    _id: 'forest',
    name: 'Magic Forest',
    description: 'A quiet forest habitat with trees, berries and fresh water.'
  },
  {
    _id: 'mountains',
    name: 'Crystal Mountains',
    description: 'A cold mountain habitat with rocks, caves and clean air.'
  },
  {
    _id: 'valley',
    name: 'Sunny Valley',
    description: 'A warm valley habitat with grass, flowers and open space.'
  }
]
learn>

```

Затем нескольким единорогам было добавлено поле `habitat` со ссылкой на документ из коллекции `habitats`. Ссылка была записана в формате DBRef: `$ref` содержит имя коллекции, а `$id` — идентификатор связанного документа.

```

db.unicorns.updateOne({name: "Aurora"}, {$set: {habitat: {$ref: "habitats", $id: "forest"}}})
db.unicorns.updateOne({name: "Unicrom"}, {$set: {habitat: {$ref: "habitats", $id:
"mountains"}}})
db.unicorns.updateOne({name: "Pilot"}, {$set: {habitat: {$ref: "habitats", $id: "valley"}}})
db.unicorns.updateOne({name: "Dunx"}, {$set: {habitat: {$ref: "habitats", $id: "forest"}}})

```



```

learn> db.unicorns.updateOne(
  {name: "Aurora"},
  {
    $set: {
      habitat: {$ref: "habitats", $id: "forest"}
    }
  }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.updateOne(
  {name: "Unicrom"},
  {
    $set: {
      habitat: {$ref: "habitats", $id: "mountains"}
    }
  }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.updateOne(
  {name: "Pilot"},
  {
    $set: {
      habitat: {$ref: "habitats", $id: "valley"}
    }
  }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.updateOne(
  {name: "Dunx"},
  {
    $set: {
      habitat: {$ref: "habitats", $id: "forest"}
    }
  }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}

```

После добавления ссылок была выполнена проверка документов единорогов, у которых появилось поле habitat.

```

db.unicorns.find(
  {habitat: {$exists: true}},
  {_id: 0, name: 1, habitat: 1}
)

```

```

learn> db.unicorns.find(
|   {habitat: {$exists: true}},
|   {_id: 0, name: 1, habitat: 1}
| )
[
  { name: 'Aurora', habitat: DBRef('habitats', 'forest') },
  { name: 'Unicrom', habitat: DBRef('habitats', 'mountains') },
  { name: 'Pilot', habitat: DBRef('habitats', 'valley') },
  { name: 'Dunx', habitat: DBRef('habitats', 'forest') }
]

```

Для проверки связи был получен документ единорога Aurora. В используемой версии mongosh ссылка отображалась как DBRef('habitats', 'forest'), поэтому для получения

идентификатора связанного документа использовалось свойство `oid`. По этому значению был найден документ зоны `forest` в коллекции `habitats`.

```
var aurora = db.unicorns.findOne({name: "Aurora"})
db.habitats.findOne({_id: aurora.habitat.oid})
```

```
learn> db.habitats.findOne({_id: aurora.habitat.oid})
{
  _id: 'forest',
  name: 'Magic Forest',
  description: 'A quiet forest habitat with trees, berries and fresh water.'
}
```

Шаг 11. Создание уникального индекса

Далее была выполнена настройка индекса по полю `name` в коллекции `unicorns`. Сначала был просмотрен текущий список индексов и выполнена проверка на наличие повторяющихся имен. Так как дубли не были найдены, можно было создать уникальный индекс.

```
db.unicorns.getIndexes()
db.unicorns.aggregate([
  {$group: {_id: "$name", count: {$sum: 1}}},
  {$match: {count: {$gt: 1}}}
])
```

```
learn> db.unicorns.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
learn> db.unicorns.aggregate([
  {
    $group: {
      _id: "$name",
      count: {$sum: 1}
    }
  },
  {
    $match: {
      count: {$gt: 1}
    }
  }
])
learn> |
```

Уникальный индекс был

создан командой `createIndex()`. Параметр `unique: true` запрещает добавлять документы с одинаковым значением поля `name`.

```
db.unicorns.createIndex({name: 1}, {unique: true})
db.unicorns.getIndexes()
```

```
learn> db.unicorns.createIndex(
|   {name: 1},
|   {unique: true}
| )
name_1
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
```

Для проверки работы уникальности была выполнена попытка добавить второго единорога с именем Aurora. MongoDB вернула ошибку duplicate key, что подтверждает корректную работу уникального индекса.

```
db.unicorns.insertOne({
  name: 'Aurora',
  loves: ['test'],
  weight: 100,
  gender: 'f'
})
```

```
learn> db.unicorns.insertOne({
|   name: 'Aurora',
|   loves: ['test'],
|   weight: 100,
|   gender: 'f'
| })
MongoServerError: E11000 duplicate key error collection: learn.unicorns index: name_
1 dup key: { name: "Aurora" }
learn> |
```

Шаг 12. Управление индексами

После создания уникального индекса был получен список всех индексов коллекции unicorns. В списке отображались стандартный индекс `_id_` и пользовательский индекс `name_1`.

```
db.unicorns.getIndexes()
```

Затем пользовательский индекс `name_1` был удален командой `dropIndex()`. После повторной проверки в коллекции остался только обязательный индекс `_id_`.

```
db.unicorns.dropIndex("name_1")
db.unicorns.getIndexes()
```

```
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_',
    { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn> db.unicorns.dropIndex("name_1")
{ nIndexWas: 2, ok: 1 }
learn> db.unicorns.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
learn> db.unicorns.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index
learn> |
```

После этого была выполнена попытка удалить индекс `_id_`. MongoDB вернула ошибку, так как индекс идентификатора является обязательным системным индексом коллекции и не может быть удален обычной командой.

```
db.unicorns.dropIndex("_id_")
```

Шаг 13. План запроса и сравнение выполнения с индексами и без индекса

Для анализа плана запроса была создана коллекция numbers, содержащая 100000 документов. Каждый документ имел поле value со значением от 0 до 99999.

```
db.numbers.drop()
for (i = 0; i < 100000; i++) {
  db.numbers.insertOne({value: i})
}
db.numbers.countDocuments()
```

```
learn> db.numbers.drop()
true
learn> for (i = 0; i < 100000; i++) {
|   db.numbers.insertOne({value: i})
| }
| {
|   acknowledged: true,
|   insertedId: ObjectId('6a3ab4c3604594cf4cad47c4')
| }
learn> db.numbers.countDocuments()
100000
learn> |
```

Затем были выбраны последние четыре документа. Для этого данные были отсортированы по полю value по убыванию и ограничены четырьмя документами.

```
db.numbers.find().sort({value: -1}).limit(4)
```

```
learn> db.numbers.countDocuments()
100000
learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a3ab4c3604594cf4cad47c4'), value: 99999 },
  { _id: ObjectId('6a3ab4c3604594cf4cad47c3'), value: 99998 },
  { _id: ObjectId('6a3ab4c3604594cf4cad47c2'), value: 99997 },
  { _id: ObjectId('6a3ab4c3604594cf4cad47c1'), value: 99996 }
]
```

До создания индекса был выполнен анализ плана запроса с помощью explain("executionStats"). MongoDB выполнила полный просмотр коллекции и сортировку результата.

```
db.numbers.find().sort({value: -1}).limit(4).explain("executionStats")
```

```

learn> db.numbers.find().sort({value: -1}).limit(4).explain("executionStats")
{
  explainVersion: "1",
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'A50753F2',
    planCacheShapeHash: 'A50753F2',
    planCacheKey: 'A50753F2',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'SORT',
      sortPattern: { value: -1 },
      memLimit: 104857600,
      limitAmount: 4,
      type: 'simple',
      inputStage: { stage: 'COLLSCAN', nss: 'learn.numbers', direction: 'forward' }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 159,
    totalKeysExamined: 0,
    totalDocsExamined: 100000,
    executionStages: {
      isCached: false,
      stage: 'SORT',
      nReturned: 4,
      executionTimeMillisEstimate: 130,
      works: 100000,
      advanced: 4,
      needTime: 100001,
      needField: 0,
      saveState: 13,
      restoreState: 13,
      isEOF: 1,
      sortPattern: { value: -1 },
      memLimit: 104857600,
      limitAmount: 4,
      type: 'simple',
      totalDataSizeSorted: 200,
      usedDisk: false,
      spills: 0,
      spilledRecords: 0,
      spilledBytes: 0,
      spilledDataStorageSize: 0,
      peakRackedMemBytes: 200,
      inputStage: {
        stage: 'COLLSCAN',
        nReturned: 100000,
        executionTimeMillisEstimate: 23,
        works: 100001,
        advanced: 100000,
        needTime: 0,
        needField: 0,
        saveState: 13,
        restoreState: 13,
        isEOF: 1,
        nss: 'learn.numbers',
        direction: 'forward',
        docsExamined: 100000
      }
    }
  }
}

```

```

queryShapeHash: 'A50753F2',
command: {
  find: 'numbers',
  filter: {},
  sort: { value: -1 },
  limit: 4,
  'lsn': 'learn'
},
serverInfo: {
  host: 'localhost',
  port: 27017,
  version: '3.3.4',
  gitVersion: 'b6b37d3a11b70b3c4133946d3a11b70b3c41'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
  internalQueryFrameworkControl: 'trySbeRestricted',
  internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
},
ok: 1
}
learn>

```

Параметр	Значение до индекса
executionTimeMillis	159
totalDocsExamined	100000
totalKeysExamined	0
nReturned	4
stage	SORT
inputStage	COLLSCAN

До создания индекса MongoDB просмотрела все 100000 документов, затем отсортировала их по полю value и только после этого вернула 4 последних документа. Это менее эффективный вариант, потому что использовался полный просмотр коллекции COLLSCAN.

После этого был создан индекс по полю value и повторно выполнен тот же запрос с explain("executionStats").

```
db.numbers.createIndex({value: 1})
db.numbers.getIndexes()
```

```
learn> db.numbers.createIndex({value: 1})
value_1
learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
learn> |
```

```
db.numbers.find().sort({value: -1}).limit(4).explain("executionStats")
```

```

learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a3ab4c3604594cf4cad47c4'), value: 99999 },
  { _id: ObjectId('6a3ab4c3604594cf4cad47c3'), value: 99998 },
  { _id: ObjectId('6a3ab4c3604594cf4cad47c2'), value: 99997 },
  { _id: ObjectId('6a3ab4c3604594cf4cad47c1'), value: 99996 }
]
learn> db.numbers.find().sort({value: -1}).limit(4).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'AE5753F2',
    planCacheShapeHash: 'AE5753F2',
    planCacheKey: 'AE69D8D4',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 4,
      inputStage: {
        stage: 'FETCH',
        nss: 'learn.numbers',
        inputStage: {
          stage: 'IXSCAN',
          nss: 'learn.numbers',
          keyPattern: { value: 1 },
          indexName: 'value_1',
          isMultiKey: false,
          multiKeyPaths: { value: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'backward',
          indexBounds: { value: [ '[MaxKey, MinKey]' ] }
        }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 0,
    totalKeysExamined: 4,
    totalDocsExamined: 4,
    executionStages: {
      isCached: false,
      stage: 'LIMIT',
      nReturned: 4,
      executionTimeMillisEstimate: 0,
      works: 5,
      advanced: 4,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      limitAmount: 4,
      inputStage: {

```

```

    inputStage: {
      stage: 'FETCH',
      nReturned: 4,
      executionTimeMillisEstimate: 0,
      works: 4,
      advanced: 4,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 0,
      nss: 'learn.numbers',
      docsExamined: 4,
      alreadyHasObj: 0,
      inputStage: {
        stage: 'IXSCAN',
        nReturned: 4,
        executionTimeMillisEstimate: 0,
        works: 4,
        advanced: 4,
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 0,
        nss: 'learn.numbers',
        keyPattern: { value: 1 },
        indexName: 'value_1',
        isMultiKey: false,
        multiKeyPaths: { value: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'backward',
        indexBounds: { value: [ '[MaxKey, MinKey]' ] },
        keysExamined: 4,
        seeks: 1,
        dupsTested: 0,
        dupsDropped: 0,
        peakTrackedMemBytes: 0
      }
    }
  },
  queryShapeHash: 'A1C8CFCE9F916AB3A6ABCC8A8B822506390F4D987582D3F926F6F2B36CA78396',
  command: {
    find: 'numbers',
    filter: {},
    sort: { value: -1 },
    limit: 4,
    '$db': 'learn'
  },
  serverInfo: {
    host: 'nod32',
    port: 27017,
    version: '8.3.4',
    gitVersion: '4b03e7daaa316c78b9bf433846dba81637d581c0'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}
learn>

```


Параметр	Значение после индекса
executionTimeMillis	0
totalDocsExamined	4
totalKeysExamined	4
nReturned	4
stage	IXSCAN
indexName	value_1

После создания индекса value_1 план запроса изменился: MongoDB использовала этап IXSCAN, то есть просмотр индекса. Затем через FETCH были получены документы, а LIMIT ограничил результат четырьмя документами. Так как сортировка выполнялась по убыванию, индекс был пройден в обратном направлении.

Запрос	executionTimeMillis	totalDocsExamined	totalKeysExamined	План выполнения
До индекса	159	100000	0	SORT + COLLSCAN
После индекса	0	4	4	LIMIT + FETCH + IXSCAN, индекс value_1

Таким образом, запрос с индексом оказался значительно эффективнее: вместо полного просмотра коллекции из 100000 документов MongoDB обратилась только к четырем нужным значениям индекса

Ответы на контрольные вопросы

Контрольные вопросы по разделу 3

1. Как используется оператор точка?

Оператор точка используется для обращения к полям вложенных объектов. Например, если в документе есть объект `mayor`, а внутри него поле `party`, то запрос к этому полю записывается как `"mayor.party"`. В работе этот способ использовался для поиска городов с независимыми мэрами и мэрами без поля `party`

2. Как можно использовать курсор?

Курсор — это результат выборки `find()`. Его можно сохранить в переменную и затем последовательно обрабатывать документы. С курсором можно использовать методы `sort()`, `limit()`, `skip()`, а также обходить результат через `forEach()` или через `hasNext()` и `next()`. В работе курсор использовался для вывода первых двух самцов единорогов после сортировки по имени

3. Какие возможности агрегирования данных существуют в MongoDB?

В MongoDB можно выполнять разные агрегированные операции: подсчет документов через `countDocuments()`, получение уникальных значений через `distinct()`, а также более сложную обработку через `aggregate()`. В `aggregate` можно группировать документы, считать количество объектов в группах, суммировать значения, фильтровать и преобразовывать данные

4. Какая из функций `save` или `update` более детально позволит настроить редактирование документов коллекции?

Более детально настроить редактирование позволяет `update`, а в современной версии MongoDB — `updateOne()` и `updateMany()`. Эти методы дают возможность задать условие выбора документа, изменить только нужные поля через операторы `$set`, `$inc`, `$unset`, `$push`, `$addToSet` и выбрать, один документ обновлять или несколько. Метод `save` проще: он либо сохраняет документ целиком, либо вставляет новый

5. Как происходит удаление документов из коллекции по умолчанию?

В старой синтаксисе MongoDB метод `remove()` по умолчанию удалял все документы, которые подходят под условие. Если условие пустое, например `remove({})`, удалялись все документы коллекции. В современной версии вместо `remove` обычно используются `deleteOne()` для удаления одного документа и `deleteMany()` для удаления всех документов, соответствующих условию

Контрольные вопросы по разделу 4

1. Назовите способы связывания коллекций в MongoDB.

В MongoDB коллекции можно связывать вручную, сохраняя идентификатор одного документа в поле другого документа. Также можно использовать DBRef, где ссылка хранится в формате \$ref и \$id, а при необходимости еще и \$db. Кроме того, MongoDB часто использует денормализованный подход: связанные данные можно хранить прямо во вложенном объекте или массиве внутри документа. В отличие от реляционных БД, такие связи не являются внешними ключами и не контролируются автоматически как ограничения целостности.

2. Сколько индексов можно установить на одну коллекцию в БД MongoDB?

Согласно методическим материалам, MongoDB позволяет установить до 64 индексов на одну коллекцию. Один индекс по полю `_id` создается автоматически для каждой коллекции.

3. Как получить информацию о всех индексах базы данных MongoDB?

Информацию об индексах конкретной коллекции можно получить методом `getIndexes()`, например `db.unicorns.getIndexes()`. В более старых версиях MongoDB также использовалась системная коллекция `system.indexes`, но в современной работе удобнее и корректнее использовать `getIndexes()` для нужной коллекции.

Вывод

В ходе лабораторной работы была установлена и проверена СУБД MongoDB, после чего были выполнены основные операции с документной базой данных. Была создана база learn и коллекции unicorns, towns, habitats и numbers. В коллекцию unicorns были добавлены документы с разными типами полей: строками, числами и массивами. Затем были выполнены запросы на выборку с фильтрацией, сортировкой, ограничением результата, проекциями и работой с массивами

Также были изучены логические операторы MongoDB: \$gte, \$lte, \$all и \$exists. С их помощью были выполнены более сложные запросы по весу, полу, предпочтениям и наличию или отсутствию поля. На примере коллекции towns была рассмотрена работа с вложенными объектами и оператором точка. Были использованы курсоры и агрегированные запросы, включая countDocuments(), distinct() и aggregate() с группировкой

В разделе редактирования данных были применены операторы \$set, \$inc, \$unset, \$push, \$addToSet и \$each. После каждой операции выполнялась проверка результата, что позволило убедиться в корректном изменении документов. Также были выполнены операции удаления данных из коллекции

В последней части работы были созданы ссылки между коллекциями unicorns и habitats, настроены индексы и проведен анализ плана запроса через explain(). Сравнение запроса до и после создания индекса показало, что индекс значительно повышает эффективность: без индекса MongoDB просмотрела 100000 документов, а после создания индекса только 4 ключа и 4 документа. Таким образом, в работе были закреплены основные практические навыки работы с MongoDB как с документно-ориентированной СУБД

